
Signal

Sistema de Mensajería E2E Mínima

X3DH + Double Ratchet + AES-256-GCM

Santiago Hoyos
Andres Chinchilla
Alexis Moreno
Marcus Rambal

Mayo 2026

1. Introducción

Signal es un prototipo de sistema de mensajería con cifrado de extremo a extremo (E2E) que usa los protocolos criptográficos X3DH (Extended Triple Diffie-Hellman) para el establecimiento de sesiones, y el Algoritmo de Doble Ratchet para el cifrado por mensaje con secreto hacia adelante. Este informe documenta la arquitectura, decisiones de implementación, análisis de seguridad y resultados de las pruebas reproducibles.

El sistema fue desarrollado como una aplicación web (React + Vite) con un servidor Node.js que nunca tiene acceso al contenido de los mensajes ni a las claves privadas de los usuarios, de manera que toda la criptografía ocurre en el navegador del cliente.

1.1 Objetivos

- Establecer un secreto compartido inicial con X3DH.
- Diseñar un mecanismo de actualización de claves mediante doble ratchet.
- Proteger la confidencialidad e integridad de los mensajes con el uso de un cifrador AEAD.
- Proponer soluciones prácticas para manejo de replays, mensajes fuera de orden y alteraciones.

2. Arquitectura del Sistema

El sistema se divide en dos capas independientes que se comunican mediante WebSocket y API REST:

2.1 Servidor (Backend)

El servidor está construido con Express.js y WebSocket. Se utiliza un principio fundamental: el servidor es ciego, solo almacena y retransmite blobs cifrados, no puede leer, modificar ni generar mensajes.

Componente	Archivo	Funcion
Key Server	routes/keys.js	Almacena bundles de claves públicas. GET consume una OPK (forward secrecy).
Message Store	routes/messages.js	Cola en memoria para mensajes offline. Se drena al reconectar por WS.
WebSocket Relay	app.js	Retransmite envelopes cifrados en tiempo real. Encola si destinatario offline.
Logger	utils/logger.js	Logs con timestamp, nivel, modulo y colores ANSI. Errores a stderr.

2.2 Cliente (Frontend)

El cliente (React + Vite) implementa toda la criptografía. Las claves privadas nunca salen del navegador ni se almacenan en estado React, sino que se mantienen en *useRef* para evitar exposición.

Archivo	Responsabilidad
crypto/x3dh.js	Protocolo X3DH: generación de keybundle, x3dhSend, x3dhReceive.
crypto/ratchet.js	Double Ratchet: initSend/Receive, ratchetEncrypt/Decrypt, MKSKIPPED.
crypto/utils.js	Helpers: base64url, hex, concatBytes, buildNonce, constantTimeEqual.
hooks/useCrypto.js	Hook React que orquesta X3DH + Ratchet. Claves privadas en useRef.
hooks/useChat.js	WebSocket: identify, message relay, heartbeat ping/pong cada 25s.
utils/logger.js	Logger de browser con CSS styling y grupos colapsables en DevTools.

2.3 Flujo de un mensaje

El ciclo completo de un mensaje desde la su envío en la interfaz de usuario hasta su entrega:

1. **Establecimiento de sesion (X3DH):** Alice obtiene el keybundle público de Bob del servidor, calcula SK mediante 4 operaciones DH + HKDF e inicializa el Double Ratchet.
2. **Cifrado (Double Ratchet):** Alice hace KDF_CK(CKs) para obtener una MK de un solo uso y cifra con AES-256-GCM usando un nonce estructurado de 96 bits. El header va en el AAD.
3. **Transmision (WebSocket):** El paquete cifrado {header, ciphertext, tag} se envía al servidor. Si Bob esta online, lo recibe inmediatamente y si está offline, queda encolado.
4. **Descifrado (Double Ratchet):** Bob detecta si el DH_PUB del header es nuevo (paso DH), verifica el tag AEAD y descifra el ciphertext con la MK derivada.

3. Protocolo X3DH

El protocolo X3DH (Extended Triple Diffie-Hellman) permite a Alice iniciar una sesión cifrada con Bob incluso si Bob está offline. El protocolo fue diseñado por Marlinspike y Reinhardt (2016) y es la base del protocolo Signal.

3.1 Tipos de claves

Clave	Tipo	Vida util	Funcion
IK (Identity Key)	x25519 + ed25519	Permanente	DH de autenticación + firma de SPK
SPK (Signed PreKey)	x25519	Mediano plazo	DH de sesión, firmada con IK ed25519
OPK (One-Time PreKey)	x25519	Un solo uso	Forward secrecy por sesión
EK (Ephemeral Key)	x25519	Una sesion	Generada por Alice, descartada tras X3DH

3.2 Las cuatro operaciones DH

Alice calcula cuatro secretos Diffie-Hellman con las claves de Bob y los concatena como material de entrada al HKDF:

```
DH1 = DH(IK_A_x25519, SPK_B) -- autenticación mutua
DH2 = DH(EK_A, IK_B) -- forward secrecy del EK efímero
DH3 = DH(EK_A, SPK_B) -- forward secrecy del SPK
DH4 = DH(EK_A, OPK_B) -- forward secrecy perfecto (opcional)

IKM = DH1 || DH2 || DH3 [|| DH4]
SK = HKDF(salt=0xFF...FF, IKM, info="Signal X3DH v1", len=32)
```

Bob realiza el mismo calculo con sus claves privadas, invirtiendo los roles en cada DH, y obtiene el mismo SK. El campo Associated Data (AD) se define como la concatenación de IK_A_pub y IK_B_pub y autentica la identidad de ambas partes en cada mensaje durante toda la sesión.

3.3 Implementación notable

- **Librería:** @noble/curves v2.2.0 -- x25519 y ed25519.
- **Salt HKDF:** 32 bytes 0xFF (según Signal, evita confusión con otros usos de HKDF).
- **OPKs:** El servidor consume y elimina una OPK por cada GET /api/keys/:userId, garantizando que ninguna OPK se reutilice.
- **SPK:** Firmada con ed25519 al generar el bundle.

4. Algoritmo de Doble Ratchet

El Double Ratchet combina un ratchet DH (que rota las claves de sesión en cada intercambio) con un ratchet simétrico (que deriva una clave única por mensaje). Esto garantiza secreto hacia adelante y recuperación de ruptura.

4.1 Funciones KDF

```
-- Ratchet DH (HKDF, 64 bytes divididos en dos mitades)
KDF_RK(RK, DH_out) -> (RK_new, CK_new)
    = HKDF(salt=RK, IKM=DH_out, info="SignalRatchet", len=64)

-- Ratchet simétrico (HMAC-SHA256)
KDF_CK(CK) -> (CK_new, MK)
    MK      = HMAC-SHA256(CK, 0x01)  -- clave de mensaje (un solo uso)
    CK_new  = HMAC-SHA256(CK, 0x02)  -- nueva chain key
```

4.2 Cifrado por mensaje (AES-256-GCM)

Cada mensaje usa una MK única derivada de la cadena simétrica. El cifrado utiliza AES-256-GCM con un nonce estructurado de 96 bits:

```
Nonce (12 bytes) = [dhRatchetStep (4B) | padding (4B) | messageNumber
(4B)]

AAD = headerToBytes(header) || AD

ciphertext || tag = AES-256-GCM(key=MK, nonce, plaintext, aad)
```

El header del mensaje se incluye en el AAD, lo que significa que cualquier modificación del header (sender_id, message_number, dh_pub, etc.) invalida el tag de autenticación. El AD del X3DH garantiza que los mensajes de una sesión no pueden ser reinyectados en otra.

4.3 Giros del ratchet DH

El ratchet DH se activa cuando el receptor detecta un nuevo dh_pub en el header del mensaje:

5. **Giro de recepción:** KDF_RK(RK, DH(DHs, DHr_nuevo)) --> RK_nuevo, CKr_nuevo.
6. **Giro de envío:** Genera nuevos DHs. KDF_RK(RK_nuevo, DH(DHs_nuevo, DHr_nuevo)) --> RK_nuevo2, CKs_nuevo.

Cada giro produce un par (RK, CK) fresco. Las cadenas anteriores son irre recuperables.

4.4 Mensajes fuera de orden (MKSKIPPED)

Si llega el mensaje $N+2$ antes que el $N+1$, el ratchet calcula y almacena la MK del $N+1$ en un mapa indexado. Cuando finalmente llega el $N+1$, se busca la MK en cache en lugar de avanzar la cadena. El límite es de 100 mensajes omitidos por giro DH, para prevenir ataques de agotamiento de memoria.

5. Análisis de Seguridad

5.1 Propiedades garantizadas

Propiedad	Mecanismo	Verificado
Confidencialidad	AES-256-GCM por mensaje	T04-T07
Autenticación	AEAD tag cubre ciphertext + header + AD	T10-T12
Integridad del header	Header incluido en AAD	T12
Secreto hacia adelante	OPKs de un uso; KDF_RK irreversible	T15
Recuperación de ruptura	Giro DH al recibir nuevo dh_pub	T05-T06
Anti-replay	Comparación msgNum vs Nr + verificación DHr	T13
Mensajes fuera de orden	MKSKIPPED cache con MAX_SKIP=100	T08-T09
Aislamiento de sesiones	AD diferente por par de participantes	T14

5.2 Modelo de amenazas

Adversario de red pasivo: observa tráfico cifrado. No puede leer contenido, por AES-256-GCM, ni inferir patrones de mensajes más allá de metadatos de tamaño y temporalización.

Adversario de red activo: puede modificar mensajes en tránsito. Cualquier modificación del ciphertext, tag o header invalida el tag AEAD.

Servidor comprometido: el servidor solo almacena claves públicas y blobs cifrados. No puede descifrar ningún mensaje ni impersonar a un usuario sin acceso a su clave privada.

Compromiso de clave de sesión: el giro DH periódico limita la exposición. Un atacante que compromete CKs en el momento T no puede descifrar mensajes anteriores (forward secrecy) ni los posteriores al siguiente giro DH (break-in recovery).

6. Detalles de Implementación

6.1 Librerías criptográficas

Se uso la suite @noble 2.2.0 para todas las operaciones criptográficas. Esta suite es auditada, sin dependencias externas, y compilable a WebAssembly para alto rendimiento en el navegador.

Paquete	Version	Uso en Signal
@noble/curves	2.2.0	x25519 (DH), ed25519 (firma SPK). Ambos exportados desde ed25519.js.
@noble/hashes	2.2.0	HKDF-SHA256 (KDF_RK, X3DH), HMAC-SHA256 (KDF_CK).
@noble/ciphers	2.2.0	AES-256-GCM (cifrado autenticado por mensaje).

6.2 Decisiones de diseño

- **useRef para claves privadas:** las claves privadas se almacenan en useRef, no en useState. Esto evita que React registre las claves en el historial de estado.
- **Inmutabilidad del estado del ratchet:** deepCloneState() clona el estado en cada operación de cifrado/descifrado. Esto previene condiciones de carrera y facilita el debugging.
- **Primer mensaje x3dh_init:** el header del primer mensaje incluye ik_pub, ek_pub y opk_id. Bob puede inicializar su sesión al recibirlo, sin interacción previa.
- **Protocolo WebSocket:** usa un handshake identify/identify_ack al conectar. Los mensajes pendientes se entregan inmediatamente tras el ack. Un heartbeat ping/pong cada 25 segundos mantiene la conexión activa a través de proxies.
- **Nonce estructurado:** codifica el paso DH y el número de mensaje, garantizando unicidad incluso si los contadores se reinician tras un giro DH.

6.3 Stack tecnológico

Capa	Tecnologia	Version
Frontend UI	React + Vite + Tailwind CSS	19, 8, 4
Criptografía	@noble/curves, hashes, ciphers	2.2.0
Backend	Node.js + Express + WebSocket	22, 4, 8
Pruebas	Node.js ESM nativo	22

7. Resultados de Pruebas

La suite de pruebas se implementó como un módulo ESM independiente que importa las librerías @noble directamente, sin dependencias de React o Vite.

7.1 Resultados

ID	Escenario	Resultado
T01	Alice y Bob derivan SK idéntico, primer mensaje descifrado	ÉXITOSO
T02	SK distinto en sesiones diferentes, OPK diferente	ÉXITOSO
T03	AD = concat(IK_A IK_B), simétrico, 64 bytes	ÉXITOSO
T04	Primer mensaje x3dh_init descifrado correctamente por Bob	ÉXITOSO
T05	Bob responde con giro DH, Alice descifra correctamente	ÉXITOSO
T06	Conversación de 6 mensajes alternados, todos correctos	ÉXITOSO
T07	4 mensajes consecutivos en la misma cadena simétrica	ÉXITOSO
T08	msg[1] llega antes que msg[0], MKSKIPPED los entrega	ÉXITOSO
T09	5 mensajes en orden [4,2,3,0,1] todos descifrados por caché	ÉXITOSO
T10	Ciphertext modificado, AES-GCM rechaza	ÉXITOSO
T11	Tag modificado, AES-GCM rechaza	ÉXITOSO
T12	Header modificado, AAD distinto, AES-GCM rechaza	ÉXITOSO
T13	Replay attack, segundo intento rechazado	ÉXITOSO
T14	Mensaje de sesión distinta, AD diferente, falla	ÉXITOSO
T15	OPK eliminada tras uso, reutilización produce SK diferente	ÉXITOSO

15 / 15 pruebas pasaron, el resultado es determinista y reproducible en cualquier entorno con Node.js 18+.

7.2 Cobertura de propiedades de seguridad

- **Correctitud del protocolo (T01-T07):** verifica que el SK se deriva correctamente, que el primer mensaje x3dh_init establece la sesión, y que el ratchet avanza correctamente en conversaciones multi-turno.
- **Resiliencia ante desorden (T08-T09):** verifica que MKSKIPPED funciona para 2 y 5 mensajes en ordenes arbitrarios.
- **Resistencia a ataques activos (T10-T15):** verifica que la modificación de cualquier componente del envelope (ciphertext, tag o header) o su reinyección son detectados y rechazados.

9. Conclusiones

Signal implementa el núcleo criptográfico de X3DH con Double Ratchet de forma correcta y verificable. El prototipo demuestra que es posible construir un sistema de mensajería E2E usando primitivas estándar (x25519, ed25519, HKDF, HMAC-SHA256, AES-256-GCM) que se encuentran disponibles en cualquier navegador.

Las 15 pruebas reproducibles confirman las propiedades de seguridad fundamentales del protocolo como lo son la correctitud del SK derivado, el secreto hacia adelante, la resistencia a modificación de mensajes, el anti-replay, el manejo de mensajes fuera de orden y el aislamiento entre sesiones.

Referencias

- [1] Marlinspike, M., Reinhardt, T. (2016). The X3DH Key Agreement Protocol. Signal Foundation.
- [2] Marlinspike, M., Perrin, T. (2016). The Double Ratchet Algorithm. Signal Foundation.
- [3] Cohn-Gordon, K., Cremers, C., Dowling, B., Garratt, L., Stebila, D. (2020). A Formal Security Analysis of the Signal Messaging Protocol. *Journal of Cryptology* 33.
- [4] Miller, P. (2023). @noble/curves, @noble/hashes, @noble/ciphers.
<https://github.com/paulmillr/noble-curves>
- [5] IETF RFC 7748. (2016). Elliptic Curves for Security (X25519). Langley, A., Hamburg, M., Turner, S.